IT4931 / IT5384 / IT4043E · SOICT · HUST

Apache Flink Internals

Lecture 12 — Streaming-first engine

Graphs · state · watermarks · checkpoints · runtime · network

Learning outcomes

1. **Trace** a Flink job from API → StreamGraph → JobGraph → ExecutionGraph
2. **Explain** operator chaining, forward links, and `keyBy` shuffles
3. **Describe** keyed/operator state and heap vs RocksDB backends
4. **Outline** checkpoints / savepoints / recovery and **external-sink EOS**
5. **Apply** event time, watermarks, strategies, and **timers**
6. **Use** DataStream vs **Table / Flink SQL** at the right layer
7. **Sketch** JobManager / TaskManager, slots, and network shuffle
8. **Compare** Flink with Spark Structured Streaming (L11)

Roadmap

1. **12.1** Components · reference job
2. **12.2** Code → graphs · chaining · physical plan
3. **12.3** State · checkpoints · recovery
4. **12.4** Time · watermarks · timers
5. **12.5** Runtime · slots · memory
6. **12.6** Network · end-to-end
7. **APIs** DataStream · Table / SQL · EOS sinks
8. **Compare** vs Spark SS (L11)

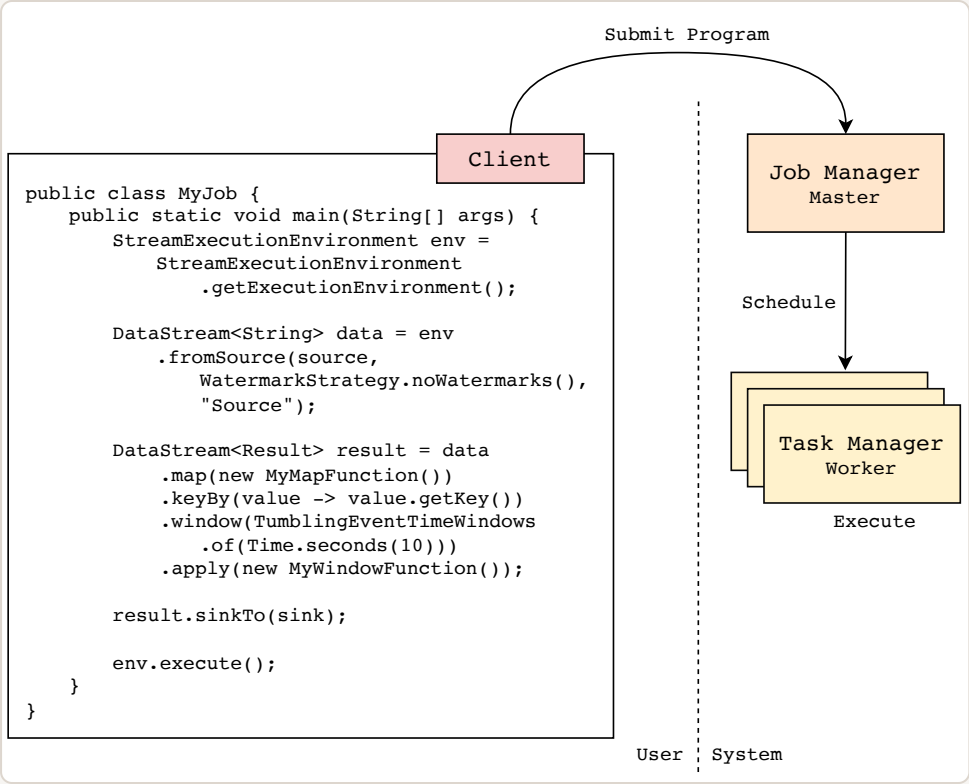
12.1 Components

User side vs system side

Side	Pieces	Role
User / Client	DataStream API · config · JAR	Compile to a graph · submit
System	**JobManager · TaskManagers**	Plan, schedule, checkpoint, run tasks

A running Flink system always has these two sides — application code never "is" the cluster.

Reference job · Client → cluster




```
public class MyJob {
    public static void main(String[] args) {
        StreamExecutionEnvironment env =
            StreamExecutionEnvironment
                .getExecutionEnvironment();

        DataStream<String> data = env
            .fromSource(source,
                WatermarkStrategy.noWatermarks(),
                "Source");

        DataStream<Result> result = data
            .map(new MyMapFunction())
            .keyBy(value -> value.getKey())
            .window(TumblingEventTimeWindows
                .of(Time.seconds(10)))
            .apply(new MyWindowFunction());

        result.sinkTo(sink);

        env.execute();
    }
}
```

Piece	Role in the figure
Client	Build graph from API · package · submit
JobManager	Accept job · schedule · coordinate checkpoints
TaskManager	Execute parallel tasks · hold local state

Nothing runs until `env.execute()` on the Client.

What the reference job does

```
// Conceptual shape (same as PyBlog MyJob)
env.fromSource(...)
   .map(...)
   .keyBy(...)
   .window(TumblingEventTimeWindows.of(Time.seconds(10)))
   .apply(...)
   .sinkTo(...);
env.execute();   // ← nothing runs until here
```

Pipeline: **Source → Map → keyBy → Window → Sink**.
We follow this single job through every internal stage.

High-level topology

```
public class MyJob {
    public static void main(String[] args) {
        StreamExecutionEnvironment env =
            StreamExecutionEnvironment
                .getExecutionEnvironment();

        DataStream<String> data = env
            .fromSource(source,
                WatermarkStrategy.noWatermarks(),
                "Source");

        DataStream<Result> result = data
            .map(new MyMapFunction())
            .keyBy(value -> value.getKey())
            .window(TumblingEventTimeWindows
                .of(Time.seconds(10)))
            .apply(new MyWindowFunction());

        result.sinkTo(sink);

        env.execute();
    }
}
```

Source

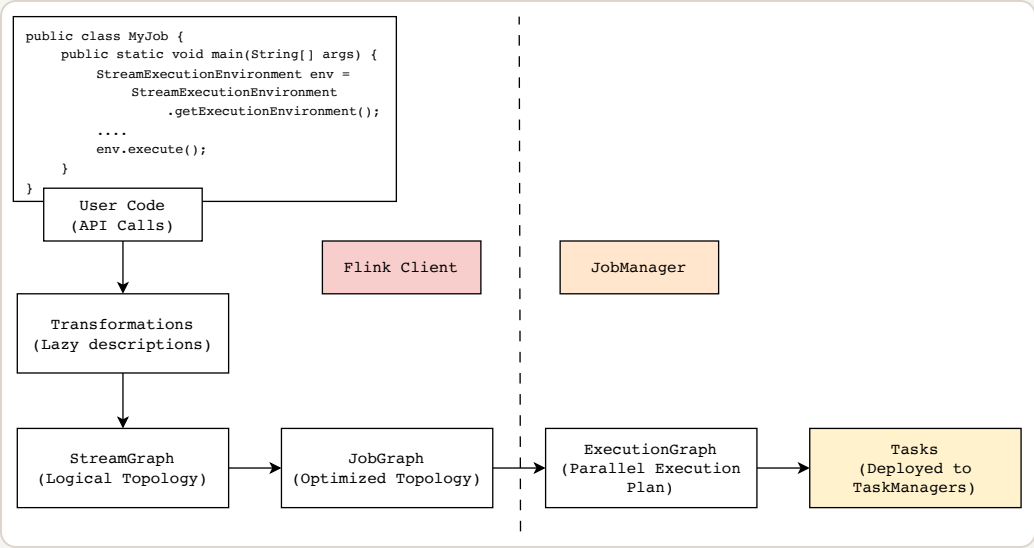
Transformations

Sink

- Operators form a **dataflow DAG** (source → transforms → sink)
- Edges carry records (and later: watermarks, checkpoint barriers)
- Parallelism means **many subtasks** of the same operator

12.2 Code to execution

Planning overview

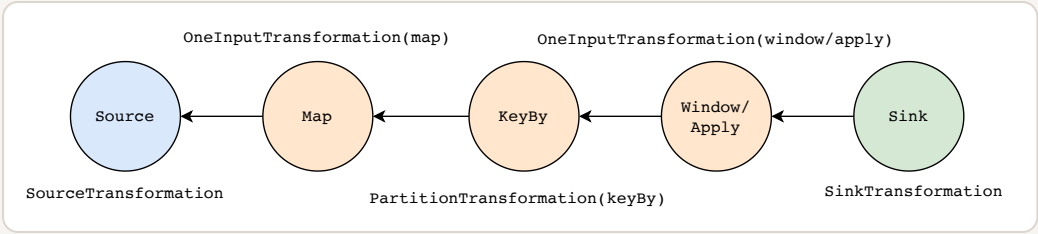


Stage	Output
API calls	List of **Transformations**
Generator	**StreamGraph** (logical)
Optimizer	**JobGraph** (chained vertices)
JobMaster	**ExecutionGraph** (parallel plan)

Compile first, distribute second — same idea every Flink job.

Transformations (before execute)

- One Transformation per API call (`fromSource` , `map` , `keyBy` , …)
- Each stores inputs, operator factory, parallelism, types
- Still **lazy**: no threads, no network yet



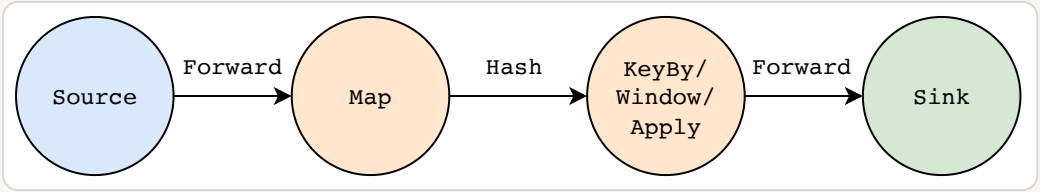
Transformation list → DAG

Each API call (`fromSource` , `map` , `keyBy` , `window` , `apply` , `sinkTo`) creates a **Transformation** on `StreamExecutionEnvironment` .

Field (conceptually)	Meaning
Input(s)	Upstream transformation(s)
Operator	User logic / factory
Parallelism	How many parallel instances
Type	Output type info

Because each points to its inputs, they form a **DAG** — still no optimization.

Logical topology · StreamGraph



Element	Meaning
StreamNode	A physical operator (Source / Map / Window / Sink)
StreamEdge	Connection + **partitioner** (forward / hash / …)
Virtual partition	`keyBy` often attaches info to the edge (no own node)

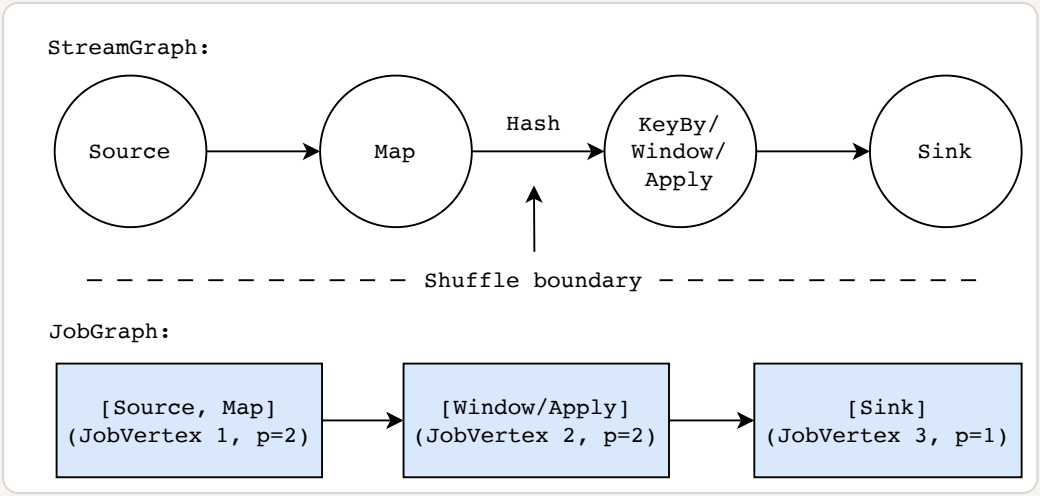
StreamGraph details

`StreamGraphGenerator` walks Transformations → **StreamGraph**.

Piece	Role
StreamNode	Physical op (Source, Map, Window, Sink)
StreamEdge	Connection + **StreamPartitioner**
Virtual nodes	e.g. `keyBy` / partition — attach info to edges, not full nodes

StreamGraph = job **logic**. Chaining / scheduling come next.

Operator chaining



- Fuse ops into **one JobVertex** → one thread, no ser/de between them
- Requires: same parallelism + **forward** edge
- `keyBy` / parallelism change → **cannot** chain (shuffle boundary)

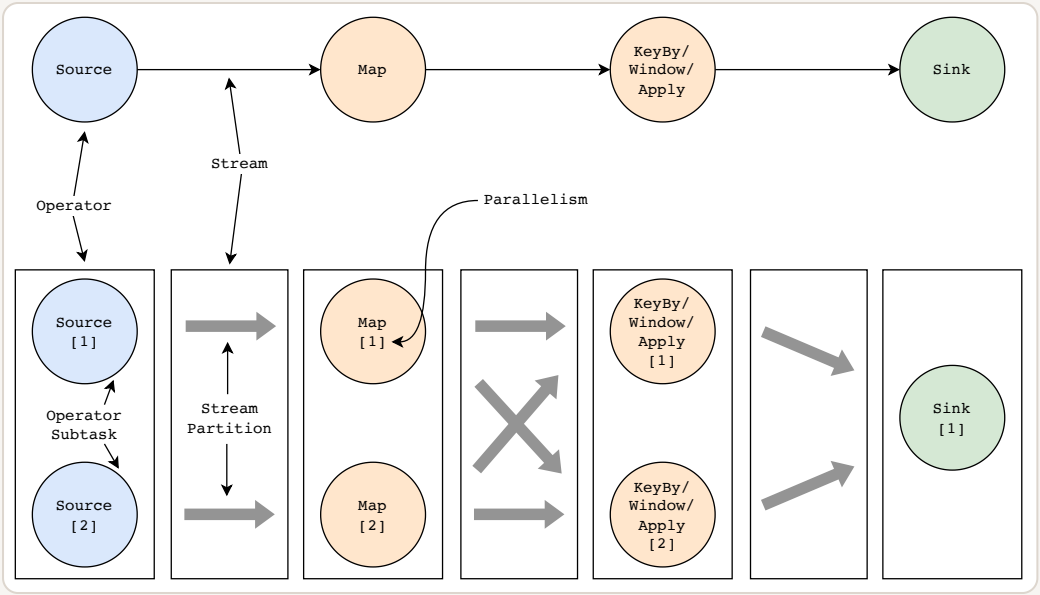
Chaining rules

`StreamingJobGraphGenerator` fuses eligible operators into one **JobVertex**.

Can chain when	Breaks chain when
Same parallelism	Different parallelism
Forward partitioner	**Hash / rebalance / broadcast** (`keyBy`)
No intervening shuffle	Shuffle boundary

Example: Source‖Map chain · Map→Window cannot (hash) · Window→Sink may not if parallelism differs (2→1).

Physical topology · forward vs shuffle



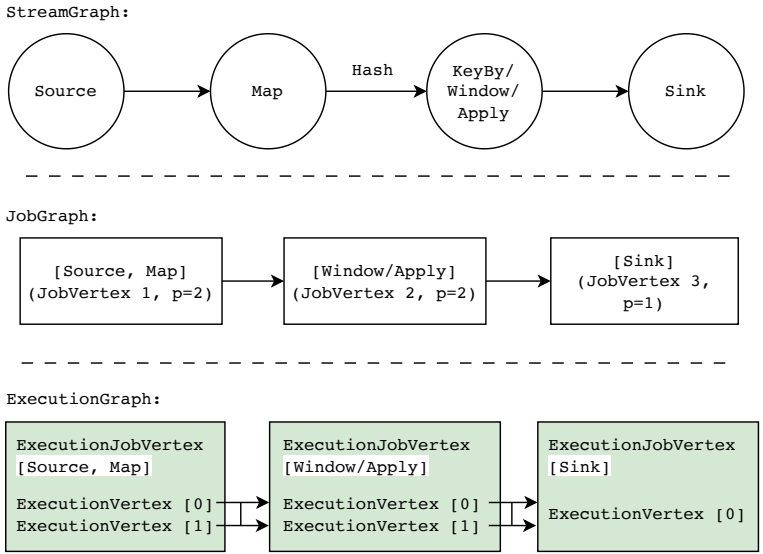
Link	Data movement	Cost
Forward	Local 1:1 partition	Cheap
Hash (keyBy)	All-to-all by key	Network + serialize
Lesson	Place shuffles carefully — they dominate latency	

Partitions & cost

Link	Behavior	Cost
Forward	1:1 local partition → partition	Cheap · no ser/de · no network
Hash shuffle (`keyBy` **)**	All upstream → all downstream by key	Expensive · serialize · TCP
Rebalance / broadcast	Other redistribution patterns	Also network

Where shuffle boundaries land is one of the biggest **performance** levers in a Flink job.

ExecutionGraph (execution plan)



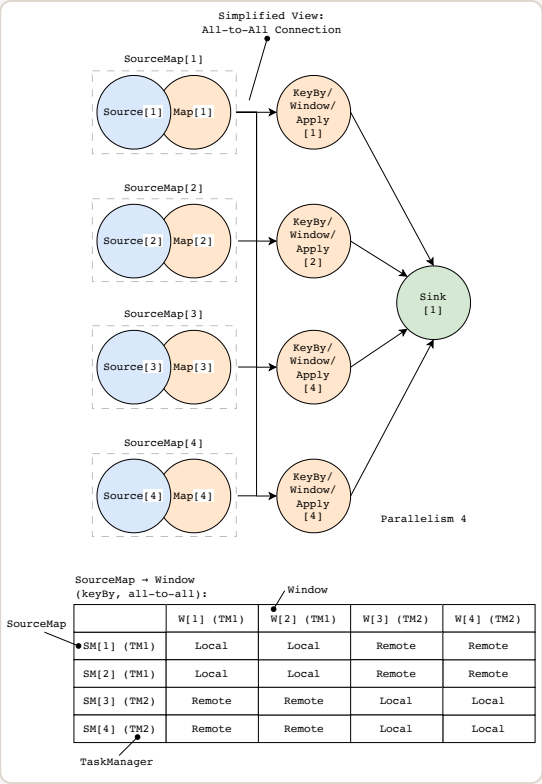
- Expand each JobVertex × **parallelism** → ExecutionVertices
- Each ExecutionVertex can have multiple **attempts** (restarts)
- Used for: scheduling, status, **checkpoint coordination**, failover

ExecutionGraph vocabulary

Term	Meaning
JobVertex	Chained operators (from JobGraph)
ExecutionJobVertex	JobVertex after expand
ExecutionVertex	One parallel subtask slot in the plan
Execution	One attempt (new attempt on restart)
Task	Deployed thread on a TaskManager

JobMaster uses ExecutionGraph for **scheduling, status, checkpoints, failover**.

Example recap (operators)



- Remember the running example: **Source → Map → keyBy → Window → Sink**
- Map is usually stateless; Window is stateful
- Everything below refers back to this pipeline

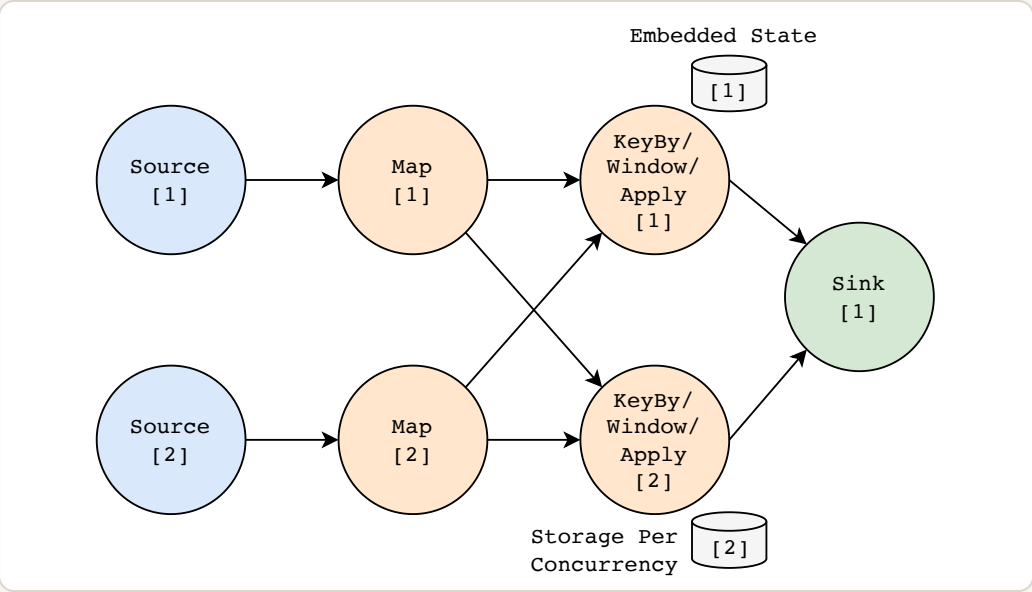
12.3 State & fault tolerance

Stateful vs stateless

Operator (example)	State?
`map`	Usually **stateless**
`window` / aggregations	**Stateful** — buffer until trigger
Sources	Often **operator state** (offsets, splits)

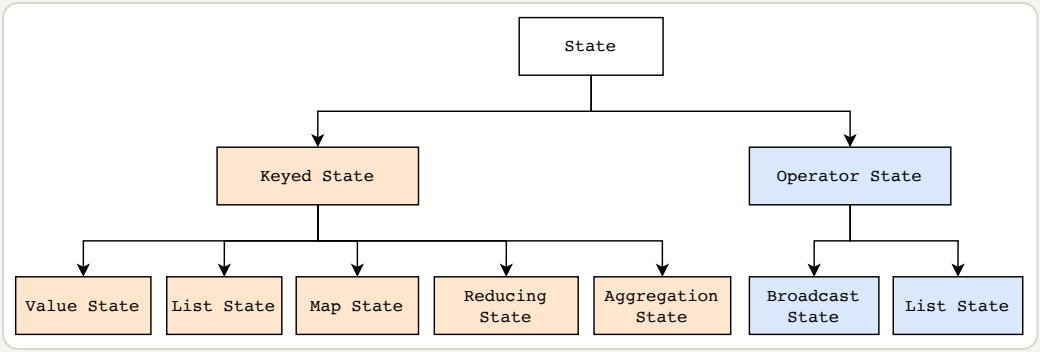
Flink state is **checkpointed** (durable) and **rescalable** (key groups) — operators do not invent their own recovery protocol.

Window operator holds state



- Incoming events for a key are **buffered** until the window fires
- Storage = local state backend on that subtask (heap or RocksDB)
- Fire condition comes from **event-time timer / watermark**

State backends



Backend	Pros	Cons
HashMap (heap)	Fast object access	Limited by memory
RocksDB	Huge state on disk	Serialize every R/W
Scope	**Per subtask** — scales with parallelism	

Backend tradeoffs

Backend	Where	Tradeoff
HashMapStateBackend	JVM heap objects	Fast · memory-bound
EmbeddedRocksDBStateBackend	Local RocksDB (disk)	Large state · serialize each access
ForStStateBackend (newer)	Remote SST (S3/HDFS) + cache	Cloud / disaggregated · async access

Each parallel **subtask** has its **own** local backend instance → state scales with parallelism.

Keyed state vs operator state

Kind	Scoped by	Examples
Keyed state	After `keyBy` · per key	`ValueState` , `ListState` , `MapState` , reducing/aggregating
Operator state	Per **subtask** (not key)	Source offsets / split assignment

Redistribution on rescale uses **Key Groups** (fixed max parallelism). Ranges of key groups move between subtasks.

Checkpointing idea

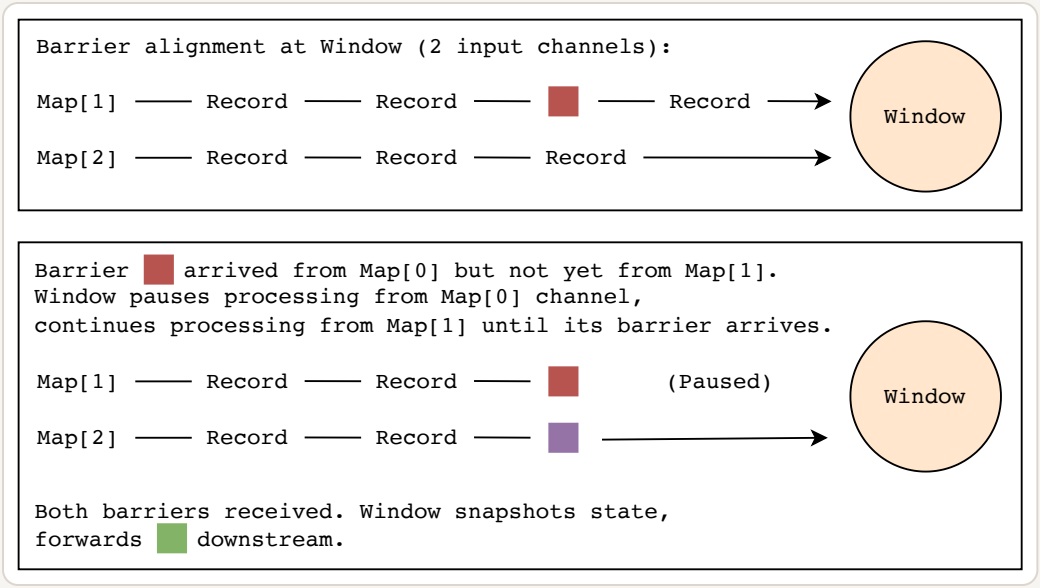
Local state is fast — but a TaskManager crash loses it.

Checkpoint = consistent global snapshot of:

1. Source positions (e.g. Kafka offsets)
2. Operator state everywhere

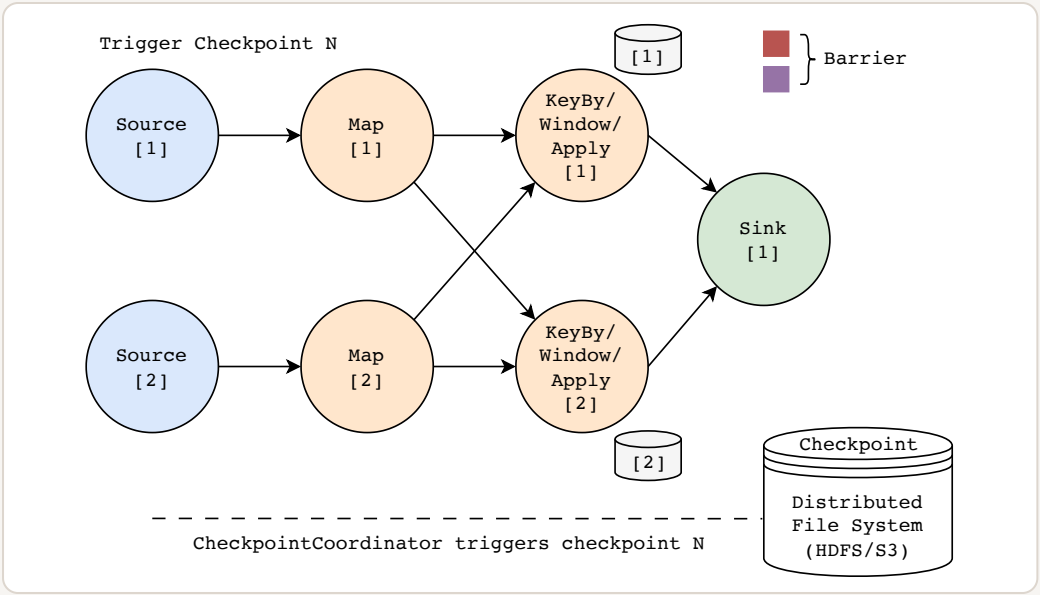
Mechanism: adapted **Chandy–Lamport** barriers flowing **in-band** with records.

Checkpoint barriers (overview)



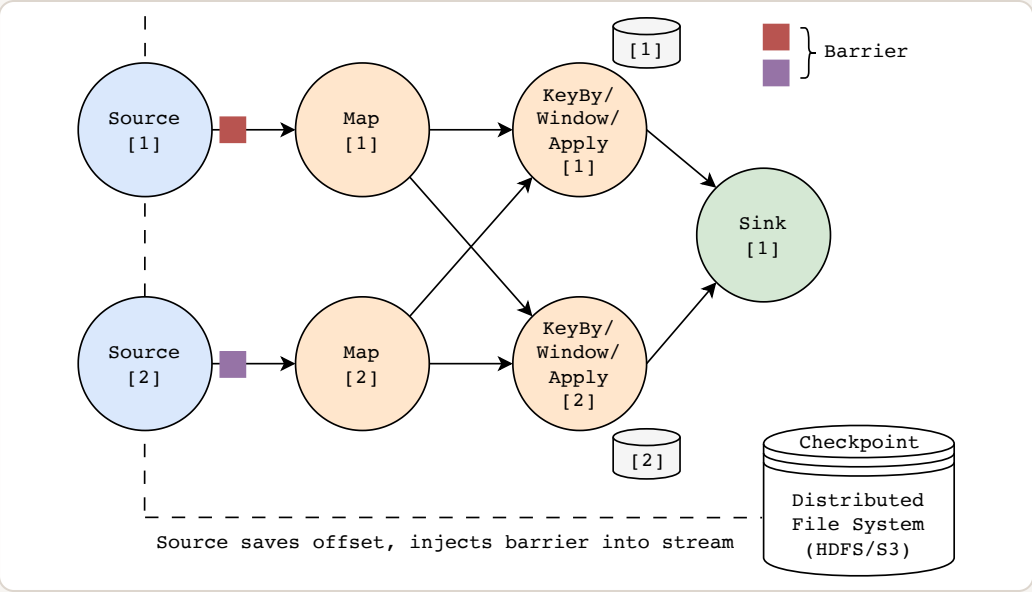
- Barriers are special records flowing **in-band** with data
- They **never overtake** normal records → consistent cut
- Snapshot = source offsets + all operator state at that cut

Checkpoint step 1



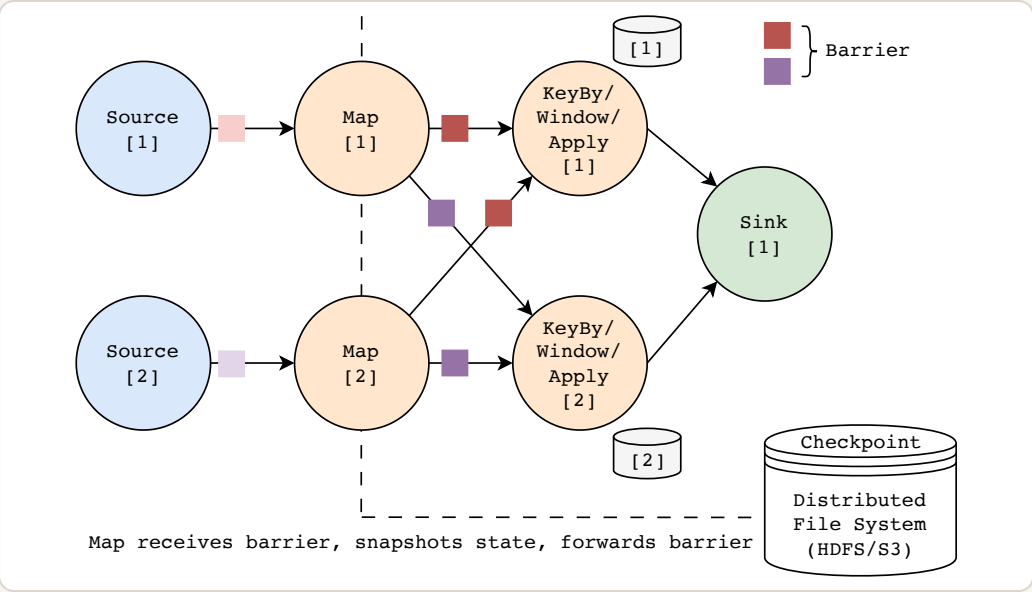
1. `CheckpointCoordinator` (JobManager) starts checkpoint *n*
2. Sends trigger to **all sources**
3. Downstream operators are not yet involved

Checkpoint step 2



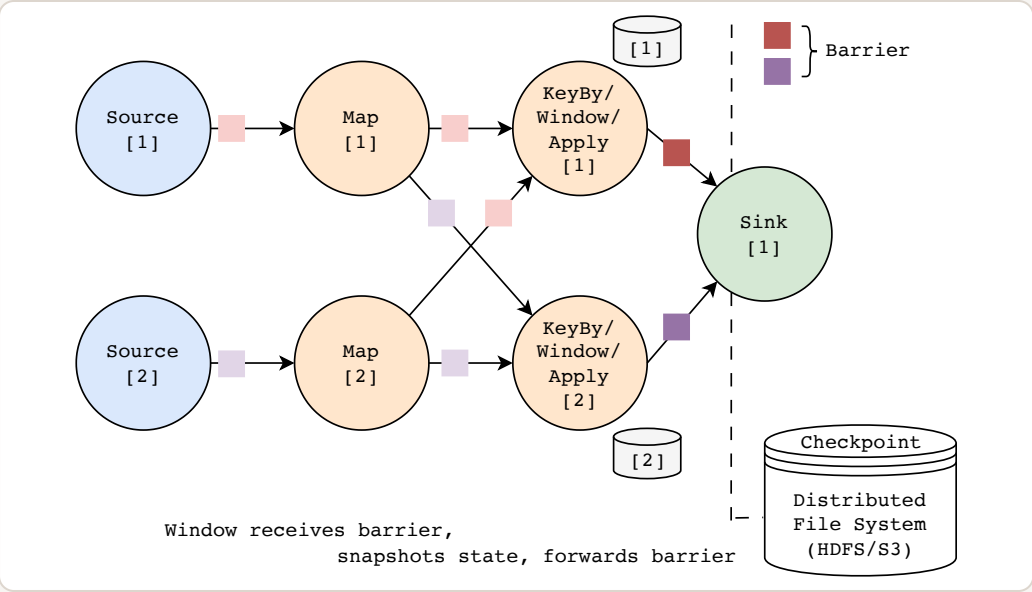
1. Each source records its **read position** (e.g. Kafka offsets)
2. Injects a **barrier** into every output channel
3. Continues emitting normal records **after** the barrier

Checkpoint step 3



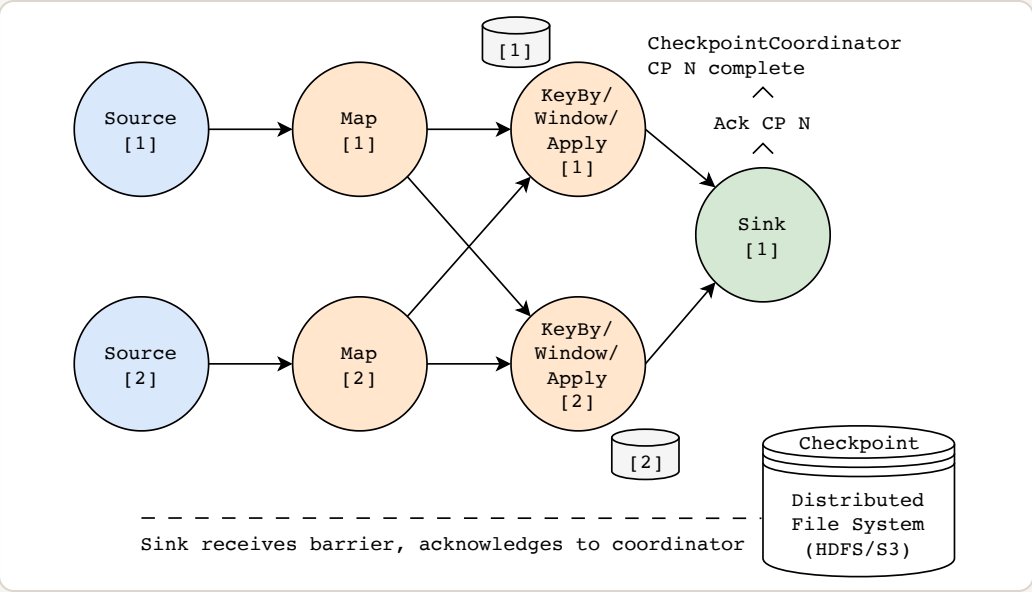
1. Operator receives barriers from inputs
2. **Aligned**: wait until **all** inputs have the barrier
3. Then snapshot local state and forward the barrier downstream

Checkpoint step 4



1. State snapshot bytes go to durable storage (HDFS / S3 / …)
2. Barriers keep propagating toward sinks
3. Async upload can overlap with continued processing (implementation detail)

Checkpoint step 5



1. Sinks receive barriers and **ACK** the coordinator
2. Checkpoint *n* marked **completed** when all ACKs arrive
3. Older checkpoints may be pruned per retention policy

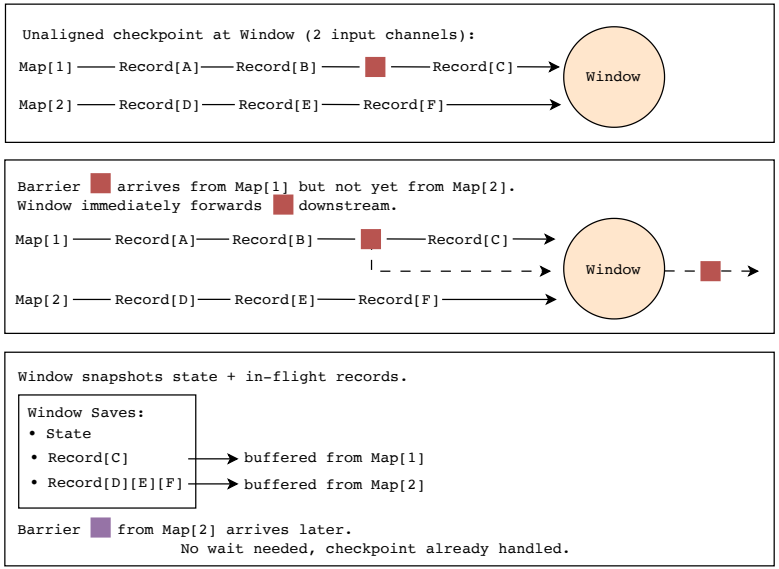
Aligned checkpoints

Multi-input operators (**after shuffle**) wait for the barrier on **all** input channels before snapshotting (**barrier alignment**).

Pros	Cons
Clean cut: state = all pre-barrier records, none after	Fast channels may **pause** → backpressure

Default mode for most jobs.

Unaligned checkpoints



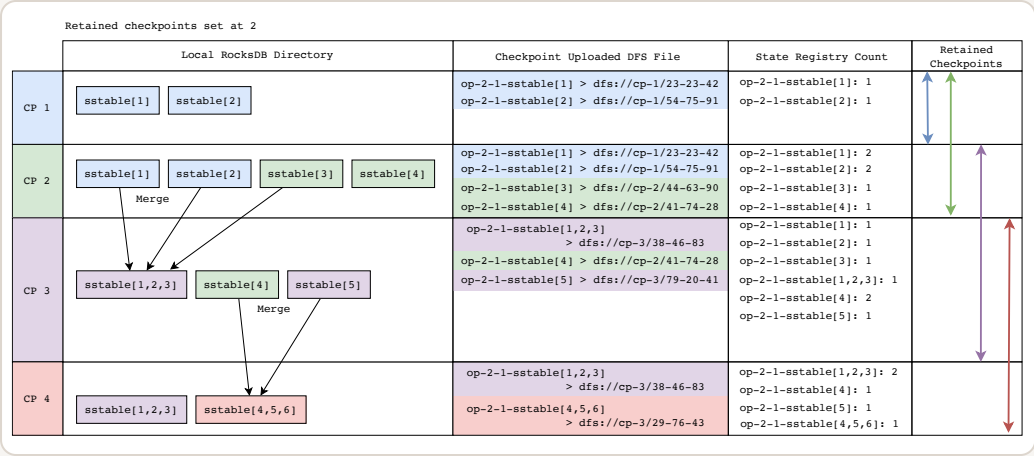
- React to the **first** barrier; do not wait to align
- Forward barrier ASAP; keep processing other channels
- Store **in-flight buffers** inside the checkpoint (larger CP)

Unaligned — when & tradeoffs

	Aligned	Unaligned
Trigger	Wait all inputs	React to **first** barrier
In-flight buffers	Not in CP	**Included** in CP state
Latency under backpressure	Can stall	CP time ≈ independent of throughput
Limits	—	Exactly-once · usually 1 concurrent CP

Hybrid: start aligned; if alignment > timeout → switch unaligned.

Incremental checkpoints (RocksDB)



- RocksDB writes immutable **SST** files
- Incremental CP uploads only **new/changed** SSTs
- Shared file registry tracks which CPs still need each file

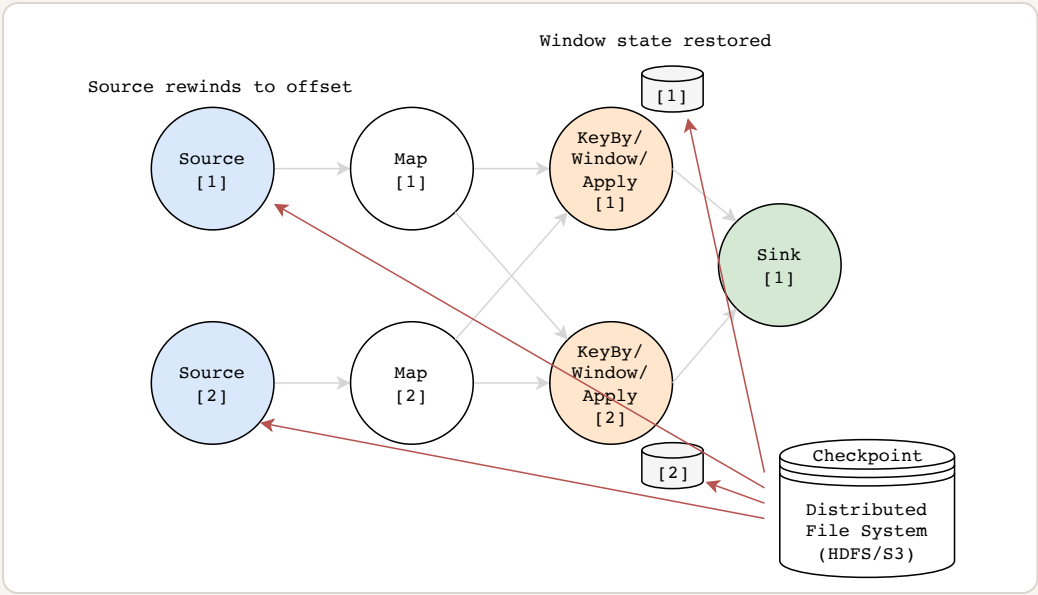
Savepoints

Same barrier machinery as checkpoints, but **ops-triggered**.

	Checkpoint	Savepoint
Trigger	Periodic	Manual (CLI / REST)
Retention	Auto-pruned	Until deleted
Alignment	Aligned or unaligned	**Always aligned**
Use	Failure recovery	Upgrade code · change parallelism · migrate cluster / backend

Canonical savepoint format can move across state backends; native format is faster but backend-tied.

Recovery · exactly-once



- Restore every operator's state from the last completed CP
- Rewind sources to stored offsets
- Replay is safe because state was rolled back → no double count **inside Flink**

Recovery steps

1. Pick latest **completed** checkpoint (all sinks ACKed)
2. Redeploy operators on available TaskManagers
3. Restore each operator's state from durable store
4. Rewind sources to checkpointed offsets
5. Reprocess records after that point on rolled-back state → **no double counting**

Requires **replayable** sources (Kafka, files, ...).

Engine EOS ≠ end-to-end with external systems

Checkpoint restore gives **exactly-once inside Flink's state**.
Writing to Kafka / JDBC / files still needs a **compatible sink protocol**.

Layer	What it guarantees
Flink checkpoint	Consistent operator state + source offsets
Idempotent / transactional sink	No duplicate side effects on replay
At-least-once sink	May duplicate external writes after failover

External sinks · two-phase commit idea

Flink's TwoPhaseCommitSinkFunction pattern (used by Kafka transactional sink, etc.):

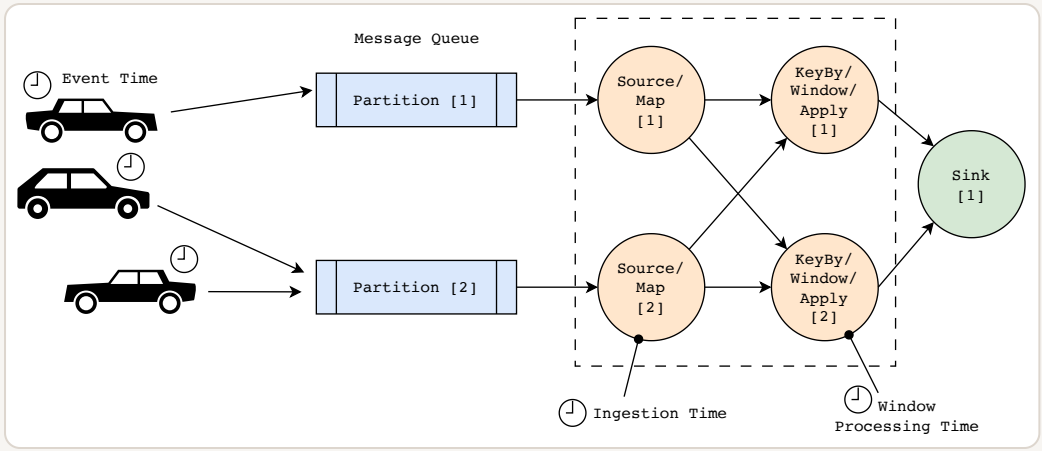
1. **Begin** transaction / open recoverable file on checkpoint start
2. Write records into the pending transaction
3. On checkpoint **pre-commit**, prepare / flush
4. On checkpoint **complete** notification → **commit** externally
5. On failure → abort / ignore uncommitted data

Sink style	Semantics on replay
Kafka **transactional**	End-to-end exactly-once (with checkpointing)
Upsert / idempotent key	Effectively EOS for that key
Plain append log	Often **at-least-once**

Ties to **L08 Kafka** transactions / idempotent producers.

12.4 Time & watermarks

Three notions of time



Time	Use
Event time	Correct windows / joins under delay
Processing time	Simple dashboards · not reproducible
Ingestion time	Rare / discouraged middle ground

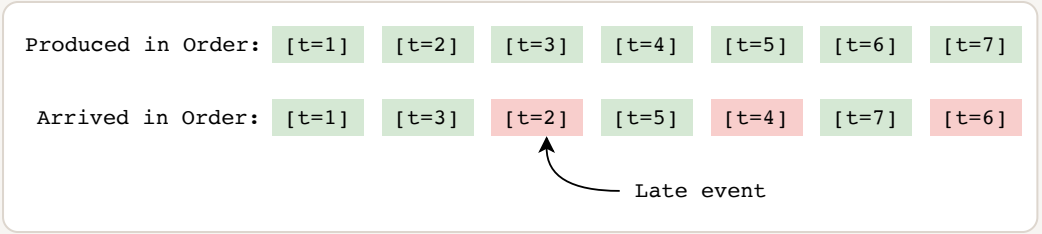
Which time to use?

Time	Meaning	Properties
Event time	Timestamp in the record	Deterministic · correct for late data
Processing time	Wall clock on the operator	Fast · non-reproducible
Ingestion time	Assign at source entry	Discouraged middle ground

Windows in the reference job use **event time** (`TumblingEventTimeWindows`).

Disorder problem

- Distributed systems reorder events
- Seeing timestamp $t+\Delta$ does **not** mean all $\leq t$ arrived
- Need a progress signal before closing windows → watermarks



Why windows cannot close early

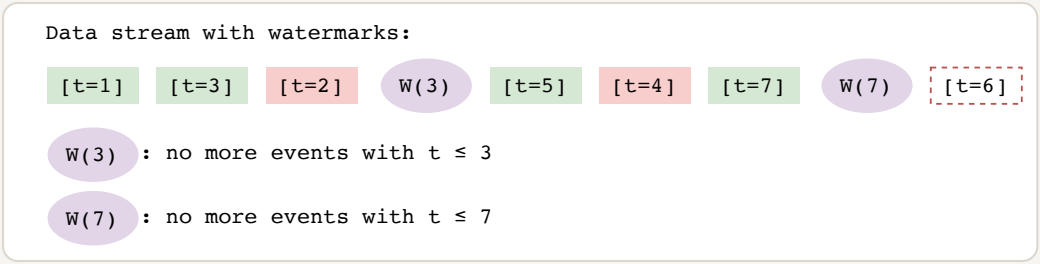
Processing time is monotonic. **Event time is not** — network, partitions, buffers reorder events.

A window `[t, t+Δ)` cannot close just because a record with timestamp `≥ t+Δ` appeared — earlier events may still be in flight.

Need a progress signal → **watermarks**.

Watermarks

- **Watermark(t)** ≈ “no more events with timestamp ≤ t”
- Windows fire when WM passes the window end
- Multi-input WM = **minimum** across channels (slowest wins)



Watermark semantics

Watermark(t) = declaration: no more events with timestamp **≤ t** (on that stream).

Effect	Detail
Window fire	When watermark passes window end
Multi-input	Effective WM = **min** across channels (slowest input wins)
Out-of-orderness gap	Larger gap → more tolerance · **higher latency** · longer state lifetime
Timers	Event-time timers fire when WM reaches registered stamp

Watermark strategies (practical)

```
// Bounded out-of-orderness — tolerate 5s disorder
WatermarkStrategy
  .<Event>forBoundedOutOfOrderness(Duration.ofSeconds(5))
  .withTimestampAssigner((e, ts) → e.getEventTime());

// Monotonous timestamps (already ordered)
WatermarkStrategy.forMonotonousTimestamps();
```

Choice	Effect
Larger bound	More late data accepted · **higher latency** · more state
Too small	Correct late events dropped as “too late”

Same idea as Spark `withWatermark`, but watermarks are **first-class stream elements**.

Timers

Operators can register **timers** that fire later:

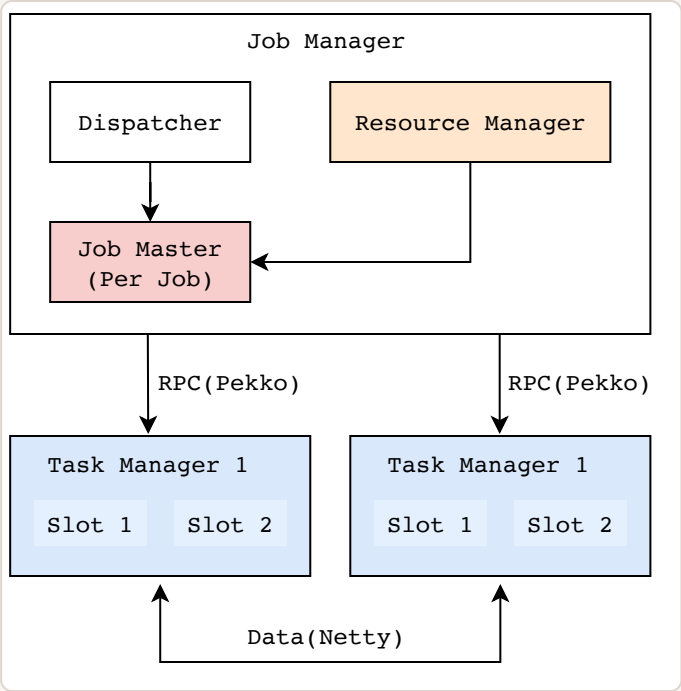
Timer kind	Fires when
Event-time timer	Watermark passes the registered timestamp
Processing-time timer	Wall clock reaches the stamp

Windows use timers internally (register end-of-window).
`ProcessFunction` / `KeyedProcessFunction` expose timers for session timeouts, delayed cleanup, periodic emit, …

```
ctx.timerService().registerEventTimeTimer(windowEnd);
// later: onTimer(...) { emit / clear state }
```

12.5 Runtime

Runtime overview



Plane	Process	Job
Control	**JobManager**	Schedule · CP · failover
Data	**TaskManager(s)**	Run Tasks · network · state

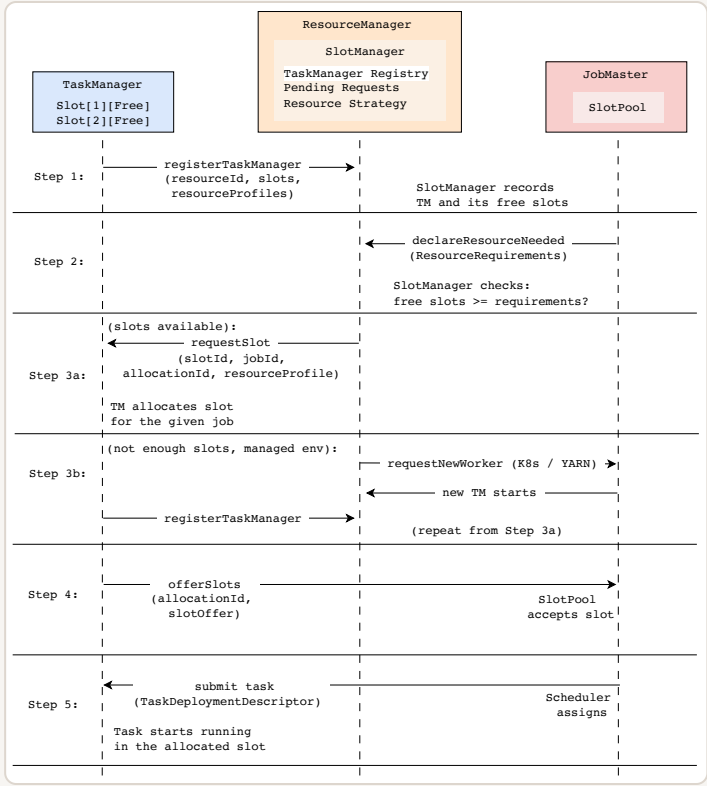
JobManager control plane

One JM process hosts several roles (same JVM):

Component	Role
Dispatcher	REST / UI · accept jobs · HA recovery of job plans
ResourceManager	Slot inventory · ask K8s/YARN for workers
JobMaster (per job)	Scheduler · **CheckpointCoordinator** · failover

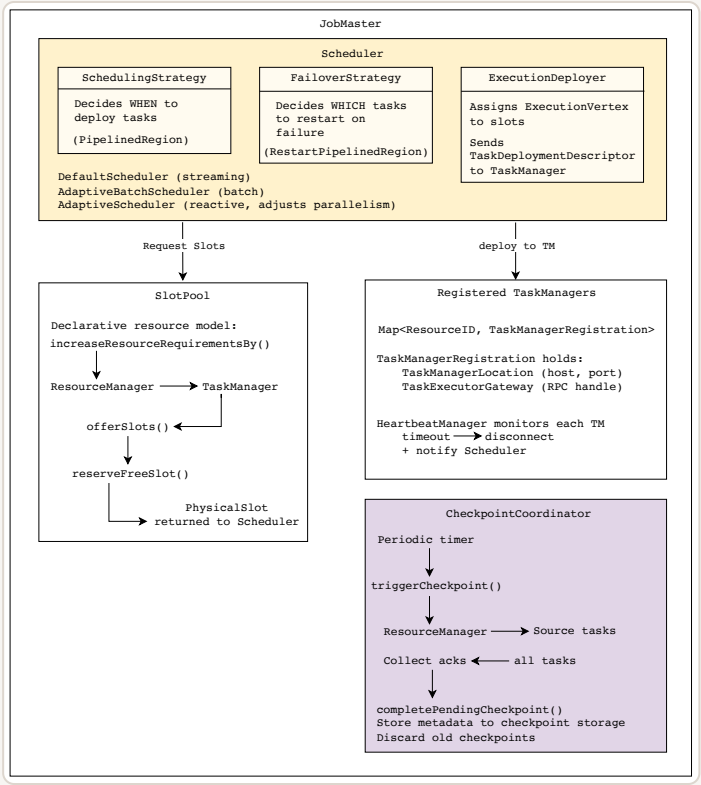
Control RPC historically Akka / now **Pekko**; data plane uses **Netty**.

Slot allocation flow



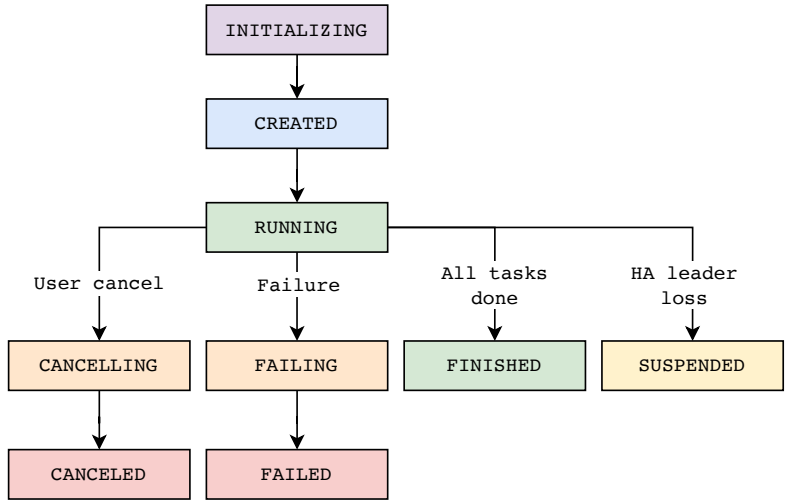
1. TaskManagers register slots with **ResourceManager**
2. JobMaster declares how many slots it needs
3. RM allocates → JM deploys ExecutionVertices into slots

JobMaster internals



Component	Responsibility
Scheduler	When/where to deploy tasks
SlotPool	Local view of granted slots
CheckpointCoordinator	Trigger · track · complete CPs

Job lifecycle



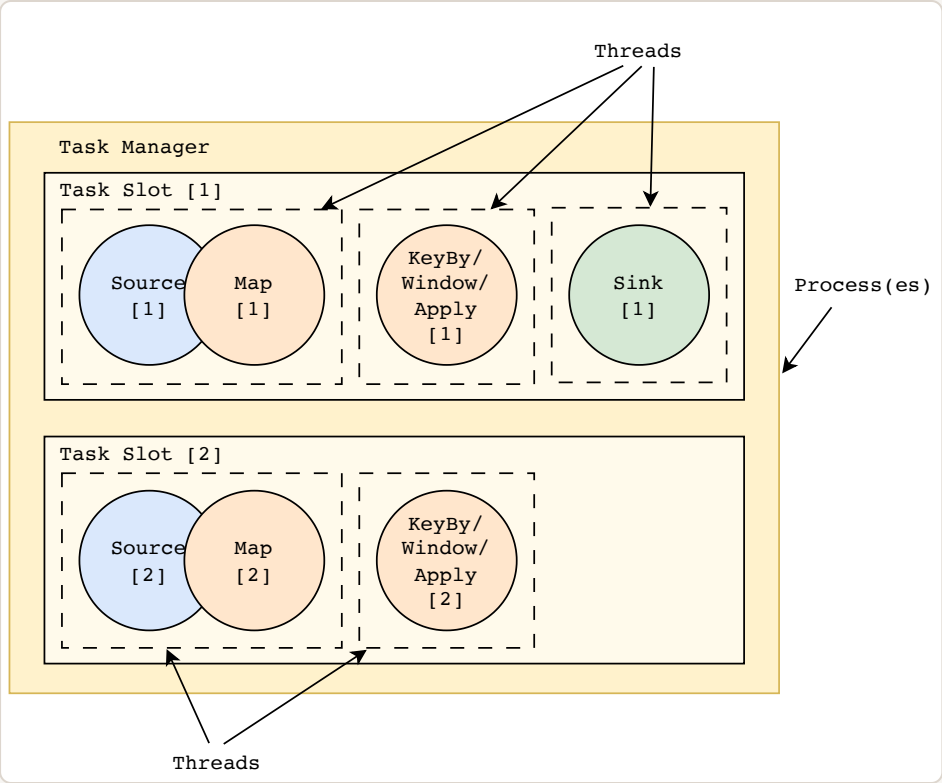
- Happy path: `CREATED → RUNNING → FINISHED`
- Failure: `RUNNING → FAILING → FAILED`
- Cancel / HA suspend have their own transitions (see next slide)

Job status (happy & sad paths)

Path	States
Success	`INITIALIZING → CREATED → RUNNING → FINISHED`
Failure	`RUNNING → FAILING → FAILED`
Cancel	`RUNNING → CANCELLING → CANCELED`
HA leadership loss	`RUNNING → SUSPENDED` (another JM may resume)

Recoverable task failures may restart a **pipelined region** while the job stays `RUNNING`.

Task slots



- Fixed number of slots per TaskManager (`taskmanager.numberOfTaskSlots`)
- A slot is a **resource slice** (CPU/memory profile)
- Multiple co-located tasks can share one slot (**slot sharing**)

Slot sharing

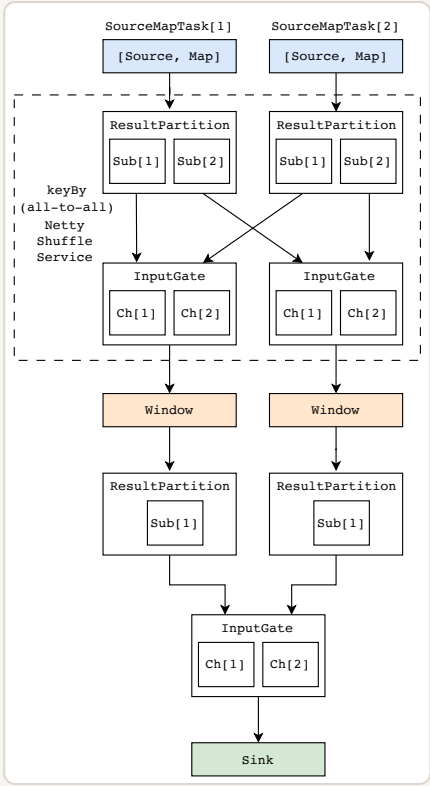
Default: all operators share one **SlotSharingGroup**.

→ One subtask from **each** pipeline stage can co-locate in a **single slot**.

Benefit	Why it matters
Fewer slots	Need ≈ **max parallelism**, not Σ parallelisms
Local forwards	Chained / forward edges stay in-process

Example: parallelism 2 across stages → **2 slots**, not 2×(#stages).

Task execution model



- One Task = one thread running an **OperatorChain**
- Reads via **InputGate**; writes via **ResultPartition**
- Sources have no InputGate — they generate data

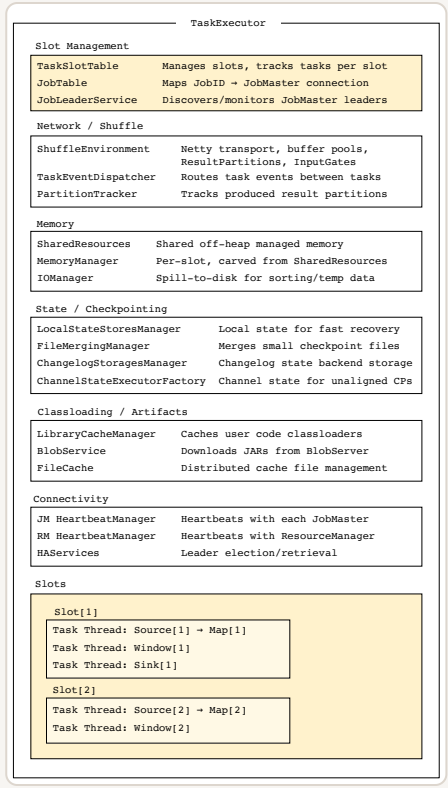
Inside a Task

```
InputGate(s)  →  OperatorChain  →  ResultPartition(s)
```

Piece	Role
InputGate	Channels from upstream subtasks
OperatorChain	Chained operators · one thread
ResultPartition	Split into subpartitions (one per downstream consumer)
Sources	No InputGate — generate records

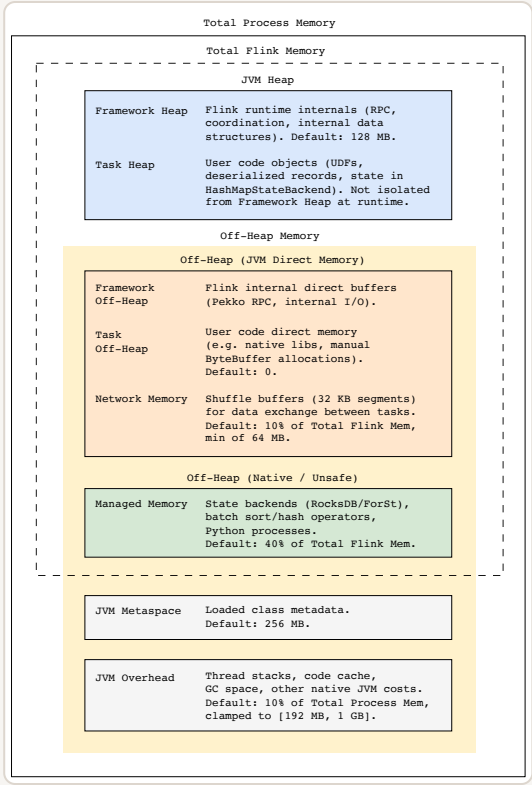
Local consumers can read without the network stack; remote uses Netty shuffle.

TaskManager services



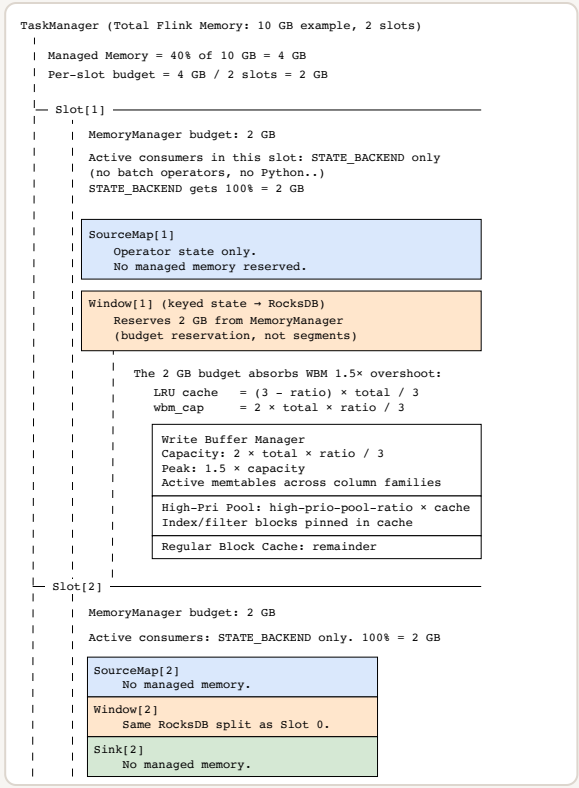
- Shared for the whole TM process: shuffle env, buffer pools, heartbeats
- State helpers: local recovery stores, changelog, file merging
- Classloaders isolate user code per job

Total process memory



- Container limit = Total **Process** Memory (Flink + JVM metaspace/overhead)
- Flink splits heap vs off-heap vs **managed** vs **network**
- Mis-sizing managed/network → OOM kill or shuffle stalls

Managed memory & RocksDB



- Managed memory budget is **per slot**
- RocksDB / sort / hash operators draw from it
- Does not fight network direct memory the same way heap does

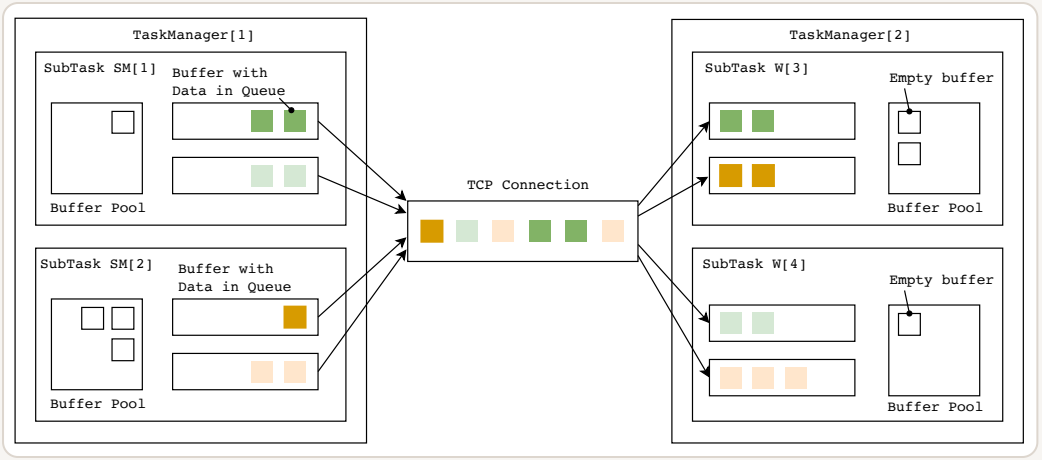
Memory takeaways

Region	Used for
Task / Framework heap	User + Flink Java objects
Network (direct)	Shuffle buffers
Managed	RocksDB / sorting / hashing (outside typical direct-memory fight)

Mis-size managed memory → container OOM kill (often **not** a catchable Java OOM).

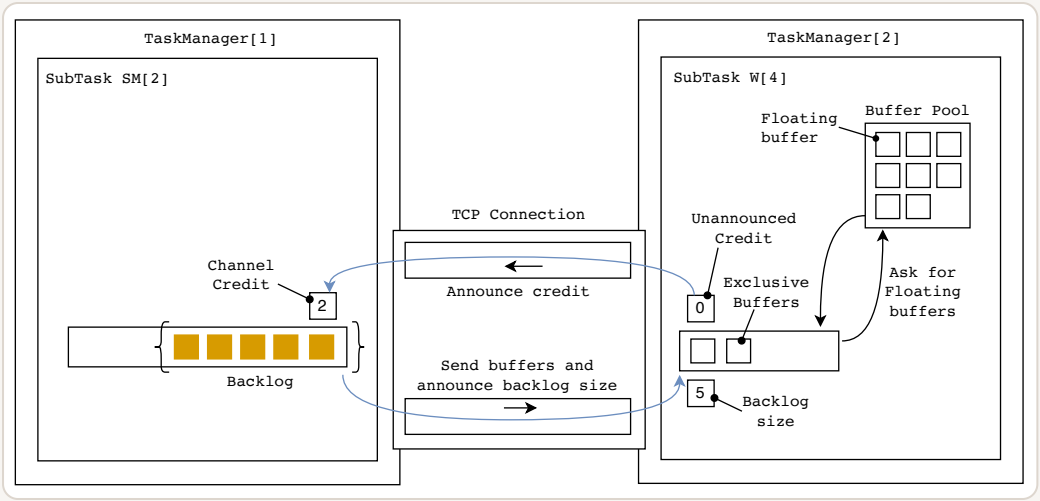
12.6 Network & wrap-up

TCP channels between TaskManagers



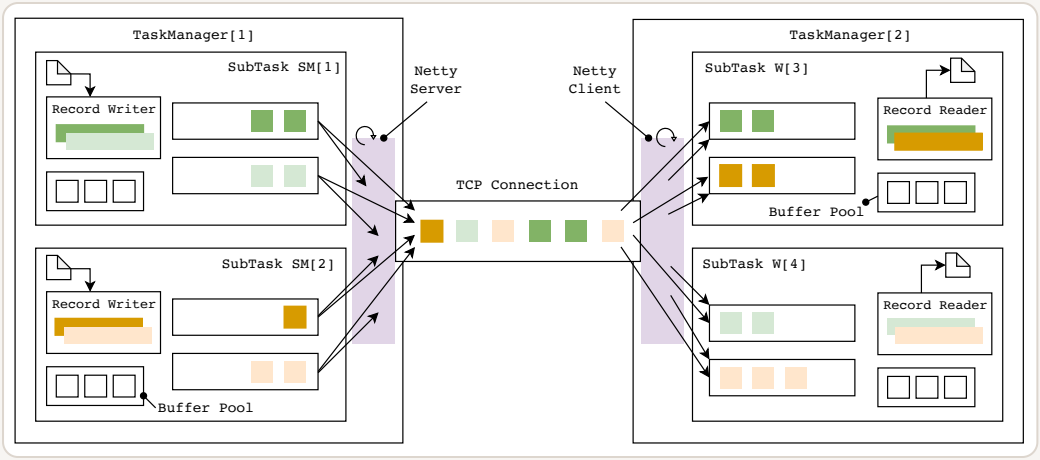
- Cross-TM shuffle = Netty over **TCP**
- Many logical subpartitions may **multiplex** one TCP connection
- Local (same TM) transfers skip the remote stack

Credit-based flow control



- Sender needs **credit** from receiver before shipping a buffer
- Backpressure stays on the slow **channel**, not the whole TCP pipe
- Helps checkpoint barriers drain faster under load

Buffer flushing

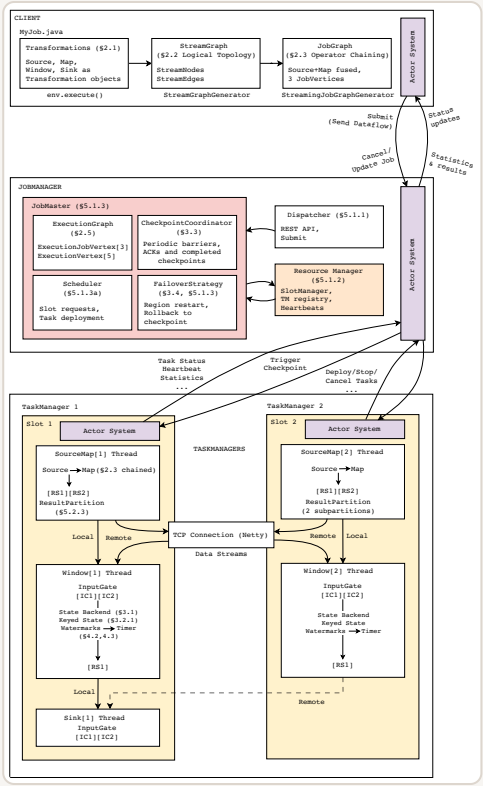


Trigger	When
Buffer full	Hand to Netty immediately
Timeout (~100ms)	Flusher wakes Netty on partial buffer
Special event	Barriers / end-of-partition flush all

Network notes

Topic	Point
Local vs remote	Same TM → local transfer; cross-TM → TCP
Multiplexing	Many subtasks → one TM pair can share channels
Backpressure	Slow downstream withholds **credits** → upstream slows
Why it matters	Shuffle + buffering dominate latency/throughput after `keyBy`

End-to-end picture



- Client → graphs → JM schedule → TM execute
- State + barriers → durable checkpoints
- Network credits/buffers keep shuffle and CPs healthy

APIs · SQL · sinks

Programming surfaces

API	Style	Use when
DataStream	Operators · `ProcessFunction` · timers	Custom logic · fine-grained state
Table API	Embedded relational DSL	Type-safe pipelines in Java/Scala/Python
Flink SQL	Declarative SQL · DDL watermarks	Analytics / CDC / many teams' default
Bridge	`fromDataStream` / `toDataStream`	Mix SQL with custom ops

This lecture's internals apply underneath **all** of these surfaces.

Flink SQL sketch

```sql
CREATE TABLE events (
  user_id BIGINT,
  tip STRING,
  ts TIMESTAMP(3),
  WATERMARK FOR ts AS ts - INTERVAL '5' SECOND
) WITH (...);

SELECT user_id,
       TUMBLE_START(ts, INTERVAL '1' MINUTE) AS w_start,
       COUNT(*) AS c
FROM events
GROUP BY user_id, TUMBLE(ts, INTERVAL '1' MINUTE);
```

Watermarks & windows in SQL map to the same runtime mechanisms you just studied.

Table API (same engine)

```
Table events = tableEnv.from("events");
Table result = events
  .window(Tumble.over(lit(1).minutes()).on($("ts")).as("w"))
  .groupBy($("user_id"), $("w"))
  .select($("user_id"), $("w").start(), $("user_id").count().as("c"));
```

Point	Detail
Unified	Same query shape on **bounded** (batch) or **unbounded** streams
Optimization	Planner → DataStream / batch exec
When	Prefer SQL/Table for relational work; drop to DataStream for custom state/timers

Spark Structured Streaming vs Flink

	Spark SS (L11)	Flink (L12)
Model	Incremental **tables**	Continuous **dataflow** + graphs
Latency	Micro-batch (sec)	Record / ms operators
Graphs	Catalyst / Spark plan	Stream/Job/**Execution** graphs
State	Query state + CP dir	Keyed/operator · heap/RocksDB
FT	WAL + versions + sinks	**Barrier checkpoints** · savepoints
Time	`withWatermark` · modes	Native WM · timers · windows

When to choose which

Spark SS — already on Spark; lakehouse + MLlib; seconds latency OK.

Flink — low-latency apps; complex keyed state/timers; dedicated stream/SQL/CDC runtime.

Course path: L11 mastery of Spark streaming semantics · **L12** depth on Flink's engine.

References

Source	Link
PyBlog — Flink Internals (primary figures)	pyblog.xyz/flink-internals
Flink architecture	nightlies …/flink-architecture
Stateful stream processing	nightlies …/stateful-stream-processing
Timely stream processing	nightlies …/concepts/time
Network stack deep dive	flink.apache.org · network stack

Figures adapted with attribution to Adesh Nalpet Adimurthy (PyBlog).

Takeaways

1. Client builds **graphs**; JM/TMs execute **Tasks**
2. **Chaining** + shuffle placement dominate performance
3. Checkpoints restore **engine state**; external EOS needs transactional/idempotent sinks
4. **Watermarks + timers** make event-time windows and custom logic safe
5. Use **SQL/Table** for relational streaming; DataStream for custom state
6. Slots, memory, and network are first-class operational concerns

Next

L13 — Spark ML / analytics